

Unity エコシステム統合

このページでは、Timeline・Animator・Visual Scripting・Input System・Networking・Scriptable Audio Pipeline・Chunity (Chuck)・Maestro / MPTK との統合機能について説明します。

名前空間:

統合	名前空間
Animator	jp.kshoji.unity.midi
Timeline	jp.kshoji.unity.midi.timeline
Visual Scripting	jp.kshoji.unity.midi.visualscripting
Input System	jp.kshoji.unity.midi.integrations.inputsystem
Networking	jp.kshoji.unity.midi.net
Scriptable Audio	jp.kshoji.unity.midi.scriptableaudio
Chunity	jp.kshoji.unity.midi.integrations.chunity
MPTK	jp.kshoji.unity.midi.mptk

配置:

統合	配置
Animator	Assets/MIDI/Scripts/Integrations/Animator/
Timeline	Assets/MIDI/Scripts/Integrations/Timeline/
Visual Scripting	Assets/MIDI/Scripts/Integrations/VisualScripting/
Input System	Assets/MIDI/Scripts/Integrations/InputSystem/
Networking	Assets/MIDI/Scripts/Integrations/Networking/
Scriptable Audio	Assets/MIDI/Scripts/Integrations/ScriptableAudio/
Chunity	Assets/MIDI/Scripts/Integrations/Chunity/
MPTK	Assets/MIDI/Scripts/Integrations/MPTK/

Unity-MCP（任意）

Cursor 等の MCP クライアントから診断・セットアップを行えます（Unity-MCP）。ツールは Assets/MIDI/Scripts/Integrations/Mcp/ にあり、com.ivanmurzak.unity.mcp 導入時（+ NuGet ゲート）のみコンパイルされます。**Asset Store** コア配布には含めません。

項目	値
サンプルプロンプト	unity-mcp-sample-prompts.md
チェックリスト	unity-mcp-define-matrix.md
Integration README	../././Scripts/Integrations/Mcp/README.md
VST ホスト	別パッケージ — Unity-VST3-Bridge

最短経路: Unity-MCP 導入 → Play Mode で `midi-init` → `midi-devices-list`。ビルド済み **Player** では `MidiMcpRuntimeBootstrap` を置き、**Host** を LAN 上の MCP Server に向け、Editor で配線済みのコンポーネントだけを制御します ([MCP README](#))。ネットワーク系は `FEATURE_MIDI_NETWORK` (`midi-net-*` の setup は Editor、`discovery-status` / `rtt` は Player)。MIDI 2.0 / MPE / CI はコア MCP asm (`midi2-*` / `mpe-zone-status` / `midi-ci-discover`)。VST ホスト MCP は [Unity-VST3-Bridge](#) のデスクトップ向け (Android 実機 MCP 対象外)。

前提条件

Timeline 統合

- Unity Package Manager で **Timeline** (`com.unity.timeline`) をインストール
- Project Settings にスクリプト定義シンボル `FEATURE_USE_TIMELINE` を追加

項目	値
Assembly Definition	<code>jp.kshoji.midi.timeline</code>
スクリプト定義シンボル	<code>FEATURE_USE_TIMELINE</code>
設定パス	<code>Project Settings > Player > Other Settings > Script Compilation > Scripting Define Symbols</code>

シンボルが未定義の場合、Timeline 統合 `asmdef` とサンプルコード (`#if FEATURE_USE_TIMELINE` ブロック内) はコンパイルされません。

Visual Scripting 統合

- Unity Package Manager で **Visual Scripting** (`com.unity.visualscripting`) をインストール
- Project Settings にスクリプト定義シンボル `FEATURE_USE_VISUALSCRIPTING` を追加

項目	値
Assembly Definition	<code>jp.kshoji.midi.visualscripting</code>
スクリプト定義シンボル	<code>FEATURE_USE_VISUALSCRIPTING</code>
設定パス	<code>Project Settings > Player > Other Settings > Script Compilation > Scripting Define Symbols</code>

シンボルが未定義の場合、Visual Scripting 統合 `asmdef` とサンプルコード (`#if FEATURE_USE_VISUALSCRIPTING` ブロック内) はコンパイルされません。

Input System 統合

- Unity Package Manager で **Input System** (`com.unity.inputsystem`) をインストール
- Project Settings にスクリプト定義シンボル `FEATURE_INPUT_SYSTEM` を追加

項目	値
Assembly Definition	<code>jp.kshoji.midi.inputsystem</code>
スクリプト定義シンボル	<code>FEATURE_INPUT_SYSTEM</code>
設定パス	<code>Project Settings > Player > Other Settings > Script Compilation > Scripting Define Symbols</code>

シンボルが未定義の場合、Input System 統合 `asmdef` とサンプルコード (`#if FEATURE_INPUT_SYSTEM` ブロック内) はコンパイルされません。

Networking 統合

- `FEATURE_MIDI_NETWORK` を追加 (UDP Hub/Client — 追加パッケージ不要)
- オプションブリッジ (パッケージは **同梱しない**) :
 - Mirror: 導入 → `FEATURE_MIRROR` (+ `MIRROR`)
 - Netcode: UPM `com.unity.netcode.gameobjects` → `FEATURE_NETCODE` (`MIDI_HAS_NETCODE`)

- WSNet2: クライアント + `WSNet2.Runtime.asmdef` → `FEATURE_WSNET2` (`MIDI_HAS_WSNET2`)。Lobby/Game サーバは別途起動

項目	値
Core アセンブリ	<code>jp.kshoji.midi.net</code>
ブリッジ	<code>jp.kshoji.midi.net.mirror</code> / <code>.netcode</code> / <code>.wsnet2</code>
スクリプト定義シンボル	<code>FEATURE_MIDI_NETWORK</code> (+ オプション)

詳細: [Networking 統合](#)・[ビルド後の処理](#)

Scriptable Audio Pipeline 統合

- Unity 6000.3 LTS 以降を使用 (WebGL 以外)
- Project Settings にスクリプト定義シンボル `FEATURE_SCRIPTABLE_AUDIO` を追加
- オプションパッケージ (本リポジトリ利用時は `Packages/manifest.json` に同梱済み) :
 - `com.unity.burst`
 - `com.unity.collections`

項目	値
Assembly Definition	<code>jp.kshoji.midi.scriptableaudio</code>
スクリプト定義シンボル	<code>FEATURE_SCRIPTABLE_AUDIO</code>
設定パス	Project Settings > Player > Other Settings > Script Compilation > Scripting Define Symbols

シンボルが未定義の場合、または Unity 6.2 以前・WebGL では統合 `asmdef` はコンパイルされません (`ScriptableAudioUtility.IsAvailable == false`)。コア `jp.kshoji.midi` は従来どおりビルド可能です。

有効化手順の詳細は [ビルド後の処理 — Scriptable Audio Pipeline 統合](#) を参照してください。

Chunity (Chuck) 統合

- Chunity を 別途 インストール (本プラグインはランタイム非同梱)
- `Integrations/Chunity/Optional/Chunity.Runtime.asmdef.example` を Chunity Scripts ルートへ `Chunity.Runtime.asmdef` としてコピー
- Project Settings にスクリプト定義シンボル `FEATURE_CHUNITY` を追加

項目	値
Assembly Definition	<code>jp.kshoji.midi.chunity</code>
スクリプト定義シンボル	<code>FEATURE_CHUNITY</code>
設定パス	Project Settings > Player > Other Settings > Script Compilation > Scripting Define Symbols

シンボルが未定義の場合、統合 `asmdef` とサンプルの `#if FEATURE_CHUNITY` ブロックはコンパイルされません。有効化手順の詳細は [ビルド後の処理 — Chunity 統合](#) を参照してください。

Scriptable Audio Generator (追加)

- Chunity Optional パッチ手順 の `useBuiltInAudioFilter` パッチを Chunity に適用
- Unity パッケージ `com.unity.collections` をインストール (`jp.kshoji.midi.chunity.scriptableaudio` が必要)
- `FEATURE_CHUNITY_SCRIPTABLE_AUDIO` を追加 (`FEATURE_CHUNITY` 必須)

項目	値
Assembly Definition	jp.kshoji.midi.chunity.scriptableaudio
パッケージ	com.unity.collections
スクリプト定義シンボル	FEATURE_CHUNITY + FEATURE_CHUNITY_SCRIPTABLE_AUDIO
サンプル	ChunityGeneratorWorkflowSampleScene

詳細: [Chunity 統合](#)

Maestro / MPTK 統合

1. Maestro / MidiPlayerTK を **別途** インストール（本プラグインはランタイム非同梱）
2. Project Settings にスクリプト定義シンボル `FEATURE_USE_MPTK` を追加
3. **Maestro Pro** の API を使う場合は、あわせて `MPTK_PRO` も追加

項目	値
Assembly Definition	jp.kshoji.midi.mptk
スクリプト定義シンボル	FEATURE_USE_MPTK （+ 任意で MPTK_PRO ）
配置	Assets/MIDI/Scripts/Integrations/MPTK/

`FEATURE_USE_MPTK` は Free 相当のブリッジ（仮想 MIDI デバイスへのシンク / MPTK を入力ソースとするアダプター）を有効にします。
`MPTK_PRO` は `#if MPTK_PRO` でガードされた Maestro Pro 依存コードを追加で有効にします。例: `OnMidiEvent` による `rewrite` パイプライン（`MptkMidiEventPipeline`）、`Writer / External` プレイヤー橋、`InnerLoop`、`ListPlayer`、`Spatializer`、および関連 Pro サンプル。
`MPTK_PRO` が無い場合、これら Pro 専用パスはコンパイルされません。`FEATURE_USE_MPTK` のみでも Free 相当の統合はビルドできます。

詳細は [Maestro / MPTK 統合](#) および [MPTK 統合](#) を参照してください。

Animator 統合

- 追加パッケージ・スクリプト定義シンボル不要（コア `asmdef jp.kshoji.midi` に含まれます）

詳細な有効化手順は [ビルド後の処理 — Unity 統合パッケージ](#) を参照してください。

Animator 統合

MIDI 入力を Unity Animator のパラメータへ自動変換します。演奏系・システム系メッセージはバインディングで写像し、SysEx / Raw UMP は `onMessage` コールバックで受け取ります。

コンポーネント

コンポーネント	役割
<code>MidiAnimatorDriver</code>	MIDI 受信 → Animator パラメータ + <code>onMessage</code>
<code>MidiAnimatorMapping</code>	ScriptableObject。バインディング定義を共有可能
<code>MidiBlendTreeDriver</code>	2-4 チャンネル CC を Blend Tree 軸へ供給

二層モデル

レイヤー	対象	用途
バインディング	Float / Int / Bool / Trigger へ写像可能な MIDI	CC、ノート、Program Change、Clock 等
<code>onMessage</code>	全メッセージ（SysEx / Raw UMP 含む）	カスタムスクリプト、ペイロード処理

`onMessage` とバインディングは同一メッセージで併用できます。二重処理を避ける場合はどちらか一方を使ってください。

バインディング（`MidiParameterBinding`）

フィールド	説明
<code>messageType</code>	<code>MidiOutgoingMessageType</code> （NoteOn / ControlChange / TimingClock 等）
<code>group</code>	0-15、-1 = 全 group
<code>channel</code>	0-15、-1 = 全チャンネル（Clock / Start 等の group 専用メッセージでは無視）
<code>controllerOrNote</code>	ノート / CC / Program 番号等
<code>valueFilter</code>	値フィルタ、-1 = すべて
<code>mapMessageValue</code>	NoteOn の velocity を Float/Int へ写像（旧 Velocity 相当）
<code>animatorParameterName</code>	対象 Animator パラメータ名
<code>parameterType</code>	Float / Int / Bool / Trigger
<code>responseCurve</code>	正規化入力 0-1 → 出力
<code>smoothingTime</code>	CC / Pitch Bend 等のスムージング秒数
<code>deviceIdFilter</code>	デバイス ID フィルタ（空 = すべて）

写像ルール

messageType	推奨 parameterType	動作
NoteOn / NoteOff	Bool / Trigger	On/Off または Trigger（On 時のみ）
NoteOn + mapMessageValue	Float / Int	velocity 正規化
ControlChange / PitchWheel / Aftertouch	Float	0-1 正規化 + カーブ
ProgramChange / SongSelect 等	Int	number を直接設定
TimingClock / Start / Stop / Reset 等	Trigger	受信時に SetTrigger
SystemExclusive / Raw UMP	—	onMessage のみ

注意: TimingClock を Trigger にバインドすると Tick ごとに発火します。Animator Trigger の消費タイミングに注意するか、onMessage で間引き処理してください。

セットアップ

- 1. Animator を持つ GameObject に MidiAnimatorDriver をアタッチ
- 2. Assets > Create > MIDI > Animator Mapping でマッピングを作成（任意）
- 3. バインディングを設定（Inspector の Message Type / Group / Channel）
- 4. SysEx / UMP が必要な場合は onMessage にリスナーを接続
- 5. シーン bootstrap で MidiManager.InitializeMidi() を呼び出す

バインディング例

用途	messageType	controllerOrNote	parameterType	パラメータ名
モーション速度	ControlChange	1	Float	Speed
ジャンプ	NoteOn	60	Trigger	Jump
打鍵強度	NoteOn + mapMessageValue	60	Float	StrikePower
音色番号	ProgramChange	5	Int	Program
再生開始	Start	—	Trigger	TransportStart

onMessage 例

```
driver.onMessage.AddListener(args =>
{
    if (args.message.messageType == MidiOutgoingMessageType.SystemExclusive)
    {
        var payload = args.message.payload;
        // SysEx 処理
    }
});
```

MidiBlendTreeDriver

XY パッドや DJ コントローラー用途向け。2-4 つの CC を指定し、対応する Animator float パラメータ（既定: BlendX / BlendY）へ 0-1 正規化値を供給します。

Timeline 統合

Unity Timeline 上で SMF 再生・MIDI 記録・マーカージェネレーションを行います。

トラックとクリップ

種類	クラス	バインディング	説明
再生トラック	MidiPlaybackTrack	SmfPlayer	SMF クリップを Timeline 時間に同期再生
再生クリップ	MidiPlaybackClip	—	MidiSequenceAsset 参照、BPM、チャンネルリマップ、mute / solo
記録トラック	MidiRecordTrack	MidiRecorder	クリップ区間中の MIDI 入力を記録
記録クリップ	MidiRecordClip	—	クリップがアクティブな間 Start/Stop 記録

セットアップ（再生）

- GameObject に PlayableDirector と SmfPlayer を配置
- Timeline ウィンドウで **Add > MIDI Playback Track** を選択
- トラックの Binding に SmfPlayer を割り当て
- トラック上に **Midi Playback Clip** を配置
- クリップの Sequence Asset に MidiSequenceAsset を設定
- Play モードで Timeline と SMF が同期再生されることを確認

時間同期

Timeline のクリップローカル時間が SmfPlayer.Seek() のマスターとなります。

- Play 中: 差分が 30ms を超えるとシークで追従
- Pause / スクラブ: SmfPlayer を一時停止し、位置をシーク

MidiPlaybackClip の outputChannel (-1 = 元チャンネル維持)、mute、solo でクリップ単位の出力制御が可能です。

制限: Timeline 上でのピアノロール編集や UMP クリップの直接編集は未対応です。

マーカージェネレーション

マーカージェネレーション	説明
MidiMarker	指定位置で MIDI メッセージを送信 (MidiTimelineNotificationReceiver 必須)
MidiSignalEmitter	Timeline Signal と同時に MIDI メッセージを送信
MidiBarMarker	小節頭。拍子情報をラベル表示
MidiTempoMarker	テンポ変更位置

MidiMarker / MidiSignalEmitter は MidiTimelineOutgoingMessage で送信内容を設定します。MIDI 1.0 演奏系・システム系に加え、SysEx / System Common / Raw UMP (MIDI 2.0) にも対応しています。

セットアップ（送信マーカージェネレーション）

- PlayableDirector と同じ GameObject に MidiTimelineNotificationReceiver を追加 (Window > MIDI > Timeline > Add Notification Receiver To Director でも可)
- Timeline の Marker Track に MidiMarker または MidiSignalEmitter を配置

3. Inspector で Message Type / Group / Channel / Payload (SysEx・UMP 時) を設定

自動生成: Project ウィンドウで `MidiSequenceAsset` を選択し、Window > MIDI > Timeline > Generate Markers From Sequence Asset を実行。Timeline ウィンドウで PlayableDirector を開いた状態で使用します。

Signal 連携

`MidiTimelineSignalHandler` を Signal Receiver のターゲットに設定し、Timeline Signal から以下を呼び出します:

- `SendNoteOn(int note, int velocity)`
- `SendNoteOff(int note, int velocity)`
- `SendControlChange(int controller, int value)`
- `SendProgramChange(int program)`
- `SendSystemExclusive(byte[] payload)`
- `SendRawUmp(uint[] umps)`
- `SendOutgoingMessage(MidiTimelineOutgoingMessage message)` — 全メッセージ種別に対応

Visual Scripting 統合

コードを書かずに Script Graph 上で MIDI 入出力を組み立てられます。

ブリッジ

`MidiVisualScriptingBridge` を Script Graph がアタッチされた GameObject に追加します。`MidiManager` からのイベントを Visual Scripting の Event Bus へ転送します。型付きイベントノードと汎用 `On MIDI Message` は併用できますが、同一メッセージに両方を接続すると二重実行になるため、どちらか一方を使ってください。

イベントノード（Events > MIDI）

ノード	出力
On MIDI Message	deviceId, messageType, group, channel, number, value, data3, payload, payloadLength — 全 MIDI 1.0 / SysEx / Raw UMP
On MIDI Note On	group, channel, note, velocity, deviceId
On MIDI Note Off	group, channel, note, velocity, deviceId
On MIDI Control Change	group, channel, controller, value, deviceId
On MIDI Program Change	group, channel, program, deviceId
On MIDI Pitch Bend	group, channel, amount (0–16383), deviceId

アクションノード

カテゴリ	ノード
MIDI > Send	Send MIDI Message （汎用 DTO + payload）、Send MIDI Note On / Off / Control Change / Program Change / Pitch Bend （いずれも <code>group</code> 対応）
MIDI > Playback	SMF Player Play / Stop
MIDI > Utility	Get Active MIDI Notes, Serialize MIDI UMP Words , Deserialize MIDI UMP Words
MIDI > Note	Note Name To Number / Number To Note Name / Is Note In Scale

Send MIDI Message の payload 例

messageType	payload
SystemExclusive	SysEx バイト列（ <code>F0 ... F7</code> ）
SystemCommonMessage	System Common バイト列
Midi2RawUmp	UMP ワード列を big-endian 4 バイト単位で連結したバイト列。 <code>Serialize MIDI UMP Words</code> ユニットでも生成可能

セットアップ

- Package Manager で Visual Scripting をインストール
- GameObject に `Script Machine` と `MidiVisualScriptingBridge` をアタッチ

3. Script Graph を作成し、**Events > MIDI** からイベントノードを配置
4. シーン bootstrap で `MidiManager.InitializeMidi()` を呼び出す
5. Play モードで MIDI 入力 → グラフが実行されることを確認

Is Note In Scale

`MidiScaleUtility` を利用。`Major` / `NaturalMinor` / `MajorPentatonic` / `MinorPentatonic` に対応。

Input System 統合

Unity Input System の Synthetic Device と `.inputactions` を使い、MIDI 入出力を既存の Input Action ベースのゲームロジックへ接続します。

コンポーネント

コンポーネント	方向	役割
MidiInputSystemBridge	MIDI → Input System	受信 MIDI を Synthetic Device へ注入。SysEx / Raw UMP は <code>onMessage</code> コールバック
InputSystemToMidiBridge	Input System → MIDI	Input Action を <code>MidiOutgoingMessage</code> として送信
MidiSyntheticDevice	—	128 ノート + 128 CC + チャンネル専用軸
MidiInputSystemMapping	—	Action 名と MIDI 条件の ScriptableObject

Synthetic Device レイアウト

コントロール	パス例	内容
<code>note[128]</code>	<code><MidiSynthetic>/note60</code>	ノート 0-1 (ベロシティ / Poly AT)
<code>cc[128]</code>	<code><MidiSynthetic>/cc1</code>	CC 0-1
<code>pitch[16]</code>	<code><MidiSynthetic>/pitch0</code>	Pitch Bend 0-1 (0.5 = センター)
<code>program[16]</code>	<code><MidiSynthetic>/program0</code>	Program Change 0-1
<code>channelPressure[16]</code>	<code><MidiSynthetic>/channelPressure0</code>	Channel Aftertouch 0-1
<code>systemPulse[16]</code>	<code><MidiSynthetic>/systemPulse0</code>	Clock / Start / Stop 等のパルス (group インデックス)

SysEx / System Common / Raw UMP はボタン・軸モデルに写像できないため Synthetic Device 非対応です。
`MidiInputSystemBridge.onMessage` で `MidiOutgoingMessage` DTO として受け取ってください。

マッピング (MIDI → Input System)

`Assets > Create > MIDI > Input System > Mapping` で `MidiInputSystemMapping` を作成します。

フィールド	説明
actionName	.inputactions 内の Action 名（ドキュメント用）
messageType	MidiOutgoingMessageType（NoteOn / ControlChange / PitchWheel 等）
group	0-15、-1 = 全 group
channel	0-15、-1 = 全チャンネル
controllerOrNote	ノート / CC 番号 / 一致条件
valueFilter	値フィルタ、-1 = すべて
targetControlIndex	CC フォールバック先（-1 = controllerOrNote）
preferDedicatedControl	true なら pitch / program / channelPressure / systemPulse を使用
invertAxis	正規化軸の反転

逆方向バインディング（Input System → MIDI）

InputSystemToMidiBridge の InputToMidiBinding は共通 DTO 形式です。

フィールド	説明
messageType / canceledMessageType	performed / canceled 時の送信種別
group / channel / number / value / data3	MidiOutgoingMessage フィールド
useActionValue	Action 値を 0-127（Pitch は 0-16383）へ変換
sendOnPerformed / sendOnCanceled	送信タイミング

旧フィールド note / controller / sendControlChange も引き続き読み込まれます。

セットアップ

1. Package Manager で Input System をインストール
2. FEATURE_INPUT_SYSTEM を定義
3. GameObject に MidiInputSystemBridge を追加し、必要なら MidiInputSystemMapping を割り当て
4. .inputactions の Binding Path に <MidiSynthetic>/note60 等を指定
5. 出力が必要な場合は InputSystemToMidiBridge を追加
6. SysEx / UMP が必要な場合は onMessage にリスナーを接続

サンプル

シーン	パス
Input System ブリッジ	Assets/MIDI/Samples/Integrations/InputSystem/Scenes/InputSystemBridgeSampleScene.unity

Scriptable Audio Pipeline 統合

Unity 6.3+ [Scriptable Audio Pipeline](#) を、MIDI タイミングと連携したリアルタイム音声生成の **オプション基盤** として提供します。参照実装としてメトロノーム Generator、DSP クロック橋渡し、Pipe 通信 DTO が含まれます。

クイックスタート

- 1. [有効化手順](#) に従い `FEATURE_SCRIPTABLE_AUDIO` を定義
- 2. 空の GameObject に **Scriptable Audio Bootstrap** (`ScriptableAudioBootstrap`) を追加
- 3. Play モード — 120 BPM のクリック音が聞こえることを確認

またはサンプルシーンを開く:

```
Assets/MIDI/Samples/Integrations/ScriptableAudio/Scenes/ScriptableAudioMetronomeSampleScene.unity
```

コンポーネント / API

型	役割
<code>ScriptableAudioBootstrap</code>	AudioSource + Generator + Bridge を一括配線
<code>ScriptableAudioUtility</code>	<code>IsAvailable</code> 、 <code>EnsureAudioSource</code> 、 <code>AttachGenerator</code> 、 <code>ValidateSetup</code>
<code>MidiMetronomeGenerator</code>	参照 <code>IAudioGenerator</code> (メトロノームクリック)
<code>MidiDspClockBridge</code>	DSP スナップショット + Pipe で Transport を Generator へ送信
<code>MidiDspClockSnapshot</code>	ゲームロジック向け読み取り専用タイミング DTO
<code>MidiDspSequenceScheduler</code>	SMF tick → DSP サンプルスケジューリング + Pipe ノートイベント
<code>MidiDspSequenceClockMode</code>	<code>SmfTempoMap</code> / <code>ExternalClock</code> (テンポ軸の排他切替)
<code>MidiDspSequenceMidiOutBridge</code>	スケジュール済みノートを <code>MidiManager</code> ヘミラー (メインスレッド送出)
<code>MidiDspUmpSequenceScheduler</code>	UMP クリップ tick → DSP サンプルスケジューリング + Pipe
<code>MidiDspUmpSequenceClockMode</code>	<code>UmpTempoMap</code> / <code>ExternalClock</code>
<code>MidiDspUmpMidi2OutBridge</code>	スケジュール済み UMP パケットを <code>MidiManager.SendMidi2RawUmp</code> ヘミラー (フレーム粒度)
<code>MidiDspUmpSequenceBootstrap</code>	AudioSource + UMP Scheduler + 参照シンセ + Clock を一括配線
<code>UmpSequenceSynthGenerator</code>	参照 <code>IAudioGenerator</code> (UMP ポリフォニック・検証用)
<code>UmpSequenceAsset</code>	<code>.midi2</code> バイナリを保持する <code>ScriptableObject</code> (<code>ToUmpSequence()</code>)
<code>MidiDspUmpSequenceSmfFallback</code>	<code>SequenceConverter</code> 経由で SMF Scheduler に委譲するフォールバック
<code>MptkDspUmpSequenceOutput</code>	UMP Out → MPTK 仮想デバイス (<code>FEATURE_USE_MPTK</code> ; <code>MPTK/ScriptableAudio/</code>)
<code>ScriptableAudioSetupReport</code>	セットアップ診断 (Error / Warning)

サンプルデモ

モード	内容
スタンドアロン	<code>MidiDspClockBridge.fallbackBpm</code> (既定 120) でクリック
外部 Clock	サンプル UI で切替。仮想デバイス <code>virtual:scriptable-audio-sample</code> から Start / Timing Clock / Stop を注入

SMF シーケンス再生（DSP 同期）

`MidiDspSequenceScheduler` が SMF の Note On/Off を Unity DSP クロック上の絶対サンプル位置にマッピングし、参照シンセ `MidiSequenceSynthGenerator` で再生します。`SmfPlayer`（フレーム駆動・MIDI デバイス出力）とは別レイヤーです。

モード	内容
SMF テンポマップ	ローカル Transport（Play/Stop/Seek）。テンポは SMF 内メタイベント + <code>tempoFactor</code>
外部 Clock	<code>clockMode = ExternalClock + MidiClockSync</code> 。Start/Stop 追従、BPM は <code>EstimatedBpm / externalClockReferenceBpm</code> で倍率
ハードウェア MIDI 出力	任意で <code>MidiDspSequenceMidiOutBridge</code> を追加。Pipe と同じスケジュールを <code>MidiManager</code> ヘミラー（フレーム粒度）

サンプル: `Assets/MIDI/Samples/Integrations/ScriptableAudio/Scenes/ScriptableAudioSequenceSampleScene.unity`

いずれも `MidiManager.InitializeMidi()` と共存します。

UMP シーケンス再生（DSP 同期）

`MidiDspUmpSequenceScheduler` が `UmpSequence`（.midi2 クリップ）の Note On/Off および UMP チャンネルボイスを Unity DSP クロック上の絶対サンプル位置にマッピングし、参照シンセ `UmpSequenceSynthGenerator` で再生します。`UmpSequencer`（ウォールクロック + 専用スレッド駆動）とは別レイヤーです。

モード	内容
UMP テンポマップ	ローカル Transport（Play / Pause / Stop / Seek）。テンポは Flex <code>Set Tempo</code> + PPQ + <code>tempoFactor</code>
外部 Clock	<code>clockMode = ExternalClock + MidiClockSync</code> 。Start/Stop 追従、BPM は <code>EstimatedBpm / externalClockReferenceBpm</code> で倍率
ハードウェア MIDI 2.0 出力	任意で <code>MidiDspUmpMidi2OutBridge</code> を追加。Pipe と同じスケジュールを生 UMP パケットとして <code>MidiManager.SendMidi2RawUmp</code> ヘミラー（フレーム粒度）
System / SysEx	<code>scheduleSystemMessages = true</code> で UMP System（type <code>0x1</code> ）をスケジュール。Data SysEx（type <code>0x3</code> ）は完了型・分割再構成とも Pipe（インライン ≤52B）および UmpOut ミラー対応

使い分け:

用途	推奨
クリップ編集・録音・.midi2 試験再生	<code>UmpSequencer</code>
ゲーム内 BGM / ループ / Seek / DSP 同期音声	<code>MidiDspUmpSequenceScheduler</code>
MIDI 1.0 SMF のみ	<code>MidiDspSequenceScheduler</code>
ハードウェア MIDI 2.0 出力（フレーム粒度で可）	<code>MidiDspUmpMidi2OutBridge</code> + <code>MidiManager.SendMidi2RawUmp</code>

サンプル: `Assets/MIDI/Samples/Integrations/ScriptableAudio/Scenes/ScriptableAudioUmpSequenceSampleScene.unity`

`MidiManager.InitializeMidi()` / `InitializeMidi2()` と共存します。UMP 出力ブリッジ利用時は `InitializeMidi2()` を呼び、MIDI 2.0 出力デバイスが接続されている必要があります。

クイックスタート（UMP）

- 1. 有効化手順に従い `FEATURE_SCRIPTABLE_AUDIO` を定義

2. 空の GameObject に **Midi Dsp Ump Sequence Bootstrap** (`MidiDspUmpSequenceBootstrap`) を追加
3. `UmpSequence` をコードで割り当てるか、サンプルシーンのデモファクトリを利用
4. Play モード — 参照シンセでノートが聞こえることを確認

```
var bootstrap = gameObject.AddComponent<MidiDspUmpSequenceBootstrap>();
bootstrap.sequence = UmpSequenceReader.ReadSequence(stream)[0];
bootstrap.EnsureReady();
bootstrap.Scheduler.Play();
```

`.midi2` をアセット化する場合は `UmpSequenceAsset` (メニュー **Assets > Create > MIDI > Import UMP Sequence Asset From File**、または `CreateAssetMenu`) を作成し、`asset.ToUmpSequence()` の結果を Bootstrap / Scheduler に渡します。詳細は [SMF ツール — UmpSequenceAsset](#) を参照してください。

サンプル UI で確認できる項目

UI	内容
C major scale (MIDI 1.0 / MIDI 2.0)	Note On/Off 再生 (MIDI 2.0 は 32-bit velocity)
テンポチェンジ (120 → 90 BPM)	Flex <code>Set Tempo</code> 区間の拍ずれがないこと
Loop first 4 notes	<code>loopStartTick</code> / <code>loopEndTick</code> (0-1920)
Tempo factor	再生速度倍率
External MIDI Clock	Start/Stop 追従
Mirror UMP packets	<code>MidiDspUmpMidi2OutBridge</code> の有効化 (トグル後にブリッジが自動作成される)
Validate Setup	<code>ScriptableAudioUtility.ValidateUmpSequenceSetup</code> 相当の診断ログ

拡張機能

機能	使い方
Group Mute / Solo	<code>MidiDspUmpSequenceScheduler.SetGroupMute(group, mute)</code> / <code>SetGroupSolo(group, solo)</code> (Group 0-15)
MIDI 2.0 制御・属性	<code>scheduleMidi2Controls</code> / <code>scheduleNativeUmpPackets</code> (CC / PC / PNC 等を Pipe または生 UMP で送出)
SMF フォールバック	Bootstrap の <code>playbackMode = SmfDelegated</code> 。 <code>SequenceConverter.ConvertUmpSequence</code> → <code>MidiDspSequenceScheduler</code>
MPTK 連携	<code>MPTK/ScriptableAudio/</code> の <code>MptkDspUmpSequenceOutput</code> (<code>FEATURE_USE_MPTK</code> + <code>FEATURE_SCRIPTABLE_AUDIO</code>) 。UMP Out を MPTK 仮想デバイスヘルレーティング

既知の制約（参照シンセ / Pipe）

項目	挙動
System（type 0x1）	Scheduler → Pipe / UmpOut には送出可能。 参照シンセは未処理 （音には出ない）
Data 128bit（type 0x5）・3-word 以上	<code>umpPacketTable</code> + UmpOut フルミラー 。Pipe インラインおよび参照シンセは 1-2 word のみ
UMP Stream（type 0xF）	対象外
ハードウェア出力の精度	フレーム粒度（内蔵シンセの DSP サンプル精度とは異なる）
WebGL / Unity 6.2 以前	統合非対応（下記「制約」と同じ）

手動確認チェックリスト（サンプルシーン）

Play Mode で `ScriptableAudioUmpSequenceSampleScene` を開き、次を確認してください。

- 1. MIDI 1.0 / MIDI 2.0 スケールが参照シンセで鳴る
- 2. テンポチェンジデモで拍がずれて蓄積しない
- 3. ループ ON で 4 音区間が繰り返される / Seek 後も再開できる
- 4. External Clock ON で仮想デバイスの Start/Stop に追従する
- 5. Validate Setup で Error がなく、UMP Out OFF 時はブリッジ欠落 Warning が出ない
- 6. （任意）UMP Out ON + `InitializeMidi2()` + 実デバイスで生 UMP が届く
- 7. （任意）実 `.midi2` を `UmpSequenceAsset` 経由で再生し、`UmpSequencer` と聴感がおおむね一致する

Edit Mode の自己検査（Pipe blittable、tick ↔ DSP、Extractor、リスク対策）は **Window > MIDI > Validate Scriptable Audio Setup** または `ScriptableAudioUtility.ValidateFoundation()` に含まれます。Unity Test Runner による自動回帰は未追加です。

実装の配置と診断 API の詳細は [Scriptable Audio 統合](#) も参照してください。

診断

- コード: `ScriptableAudioUtility.ValidateSetup(gameObject)` → `ScriptableAudioSetupReport.ToSummary()`
- UMP 専用: `ScriptableAudioUtility.ValidateUmpSequenceSetup(gameObject)`
- エディタ: **Window > MIDI > Validate Scriptable Audio Setup**（選択中の GameObject、未選択時はシーン内の最初の Bootstrap を検証。メトロノーム / SMF / UMP を自動判別）

制約

- **WebGL**: 統合 `asmdef` 除外。ビルドエラーにならず、API は `IsAvailable == false`
- **Unity 6.2 以前**: ソースガードにより非コンパイル
- **同一出力への二重駆動**: 同じ楽曲出力に対して `UmpSequencer` と `MidiDspUmpSequenceScheduler` を同時に使わない（意図的な場合を除く）
- 詳細: [プラットフォーム — Scriptable Audio / Scriptable Audio 統合](#)

Chunity (Chuck) 統合

外部導入した [Chunity](#) 上の Chuck パッチを、本プラグインの MIDI 入出力・SMF・Clock・Timeline / Visual Scripting と接続する **オプション統合** です。Chuck 内蔵 `MidiIn` は OS デバイス番号に依存し、本プラグインの仮想デバイス・フィルタ・ネットワーク経路を通らないため、連携の本丸は **Unity MIDI** → **Chuck グローバル変数 / Event** です。

公式 API リファレンス: [Chunity Documentation](#)

クイックスタート

1. [有効化手順](#) に従い Chunity と `FEATURE_CHUNITY` を用意する
2. `ChuckMainInstance` （任意で `ChuckSubInstance` ）を配置
3. 同じ GameObject （または参照先）に `MidiChuckPatchHost` — `.ck` / インラインパッチを割り当て
4. `MidiChuckBridge` を追加 — 既定では **Use Default Convention** をオン
5. Play モードで MIDI （実機または仮想 Inject）を送り、パッチが応答することを確認

コンポーネント

コンポーネント	役割	Phase
MidiChuckInstanceTarget	Main / Sub ディスパッチ (String / 配列 / 連想配列 / RunFile args / ListenOnce 等)	A
MidiChuckBridge	MIDI → SetInt / SetFloat / Array / Associative + SignalEvent	1, A
MidiChuckPatchHost	インライン / TextAsset / StreamingAssets の .ck 実行 (Context Menu「Run Patch Now」)	1
MidiChuckMapping	MIDI 条件 → Chuck グローバル / Event (AssociativeInt / AssociativeFloat 含む)	1, A
MidiChuckUtility	デフォルトグローバル名と IsAvailable / SetupHint / SetChuckLogLevel	1, E
MidiChuckEventToMidi	Chuck Event → UnityEvent / MIDI OUT (スカラー Get + 配列読取 arraySource)	2, D
MidiChuckSmfLink	SmfPlayer → 仮想デバイス (既定 virtual:chuck-smf) → Bridge	2
MidiChuckClockSync	MidiClockSync → bpm / beat / bar / transport Event	2
MidiChuckPolyVoiceHelper	ポリフォニック voiceId (+ 任意で activeNotes[] 等)	2
MidiChuckFloatSyncer / MidiChuckIntSyncer / MidiChuckStringSyncer	Chuck*Syncer ラッパ (Inspector からグローバル名指定)	B
MidiChuckFloatArraySyncer / MidiChuckIntArraySyncer	配列 Syncer ラッパ	B
MidiChuckParameterPoller	ParameterBinder の読み取り対 (多パラメータポーリング)	B
MidiChuckHostAdvancer	Unity マスター timeStep / pos / tick Event ブリッジ	C
MidiChuckPatchLifecycle	協調 stop Event + shred cleanup + 任意 Restart	C
MidiChuckSampleBank	Program / Note / Bank → サンプルバス → SetString + play Event	D
MidiChuckPreset	float/int スナップショット (+ 任意パッチ)	3
MidiChuckParameterBinder	プリセット適用・ライブ SetFloat / SetInt	3
MidiChuckMarker 等	Timeline マーカー / パラメータクリップ (Phase E アクション含む)	3, E (FEATURE_USE_TIMELINE)

Chuck VS Units	Set/Get String・配列、RunFile、Event listen、HostAdvancer、PatchLifecycle、SampleBank 等	3, E (<code>FEATURE_USE_VISUALSCRIPTING</code>)
----------------	---	--

デフォルト規約（マッピング未一致時）

MIDI	Chuck
Note On	<code>midiNote</code> , <code>midiVelocity</code> , Event <code>noteOn</code>
Note Off	<code>midiNote</code> , Event <code>noteOff</code>
CC	<code>ccNumber</code> , <code>ccValue</code> , Event <code>controlChange</code> （任意で <code>cc[]</code> ）
Pitch Bend	<code>pitchBend</code> （-1..1）
Program Change	<code>program</code>

サンプル `.ck` 側の最小契約例:

```
global Event noteOn;
global Event noteOff;
global int midiNote;
global float midiVelocity;

SinOsc osc => dac;
while (true) {
    noteOn => now;
    Std.mtof(midiNote) => osc.freq;
    midiVelocity => osc.gain;
}
```

マッピング（`MidiChuckMapping`）

`MidiChuckBinding` で「どの MIDI 条件がどの Chuck 変数／Event に対応するか」を宣言します。`messageType` / `group` / `channel` / `controllerOrNote`（いずれも `-1` = すべて）と、書き込み先（`int` / `float` / `Event` / `float` 配列）・`floatScale` ・ `broadcastEvent` を組み合わせます。コード変更なしで別 `.ck` に差し替え可能です。

Phase 2-3 の使い分け

用途	推奨
SMF → Chuck	<code>MidiChuckSmfLink</code> で仮想デバイスを揃え、 <code>SmfPlayer.outputDeviceId</code> と Bridge の <code>deviceIdFilter</code> を一致させる
外部 Clock	<code>MidiChuckClockSync</code> （BPM → <code>bpm</code> 、拍 / 小節 / Start-Stop Event）
ポリフォニー	<code>MidiChuckPolyVoiceHelper</code> + Bridge の <code>useDefaultConvention = false</code> （単音規約との二重発火を防ぐ）
プリセット UI	<code>MidiChuckPreset</code> + <code>MidiChuckParameterBinder</code> （サンプル: <code>ChunityPresetsSampleScene</code> ）
Chuck → MIDI OUT	<code>MidiChuckEventToMidi</code>

Timeline / Visual Scripting (Phase 3 / E)

追加シンボル	Assembly	内容
FEATURE_USE_TIMELINE	jp.kshoji.midi.chunity.timeline	MidiChuckMarker (SetString / SetArray / RunFile / HostAdvancer / Lifecycle* / SampleBank 等) / MidiChuckTimelineNotificationReceiver / MidiChuckParamTrack
FEATURE_USE_VISUALSCRIPTING	jp.kshoji.midi.chunity.visualscripting	カテゴリ MIDI / Chunity のユニット (Phase E: Get* / RunFile / Event listen / HostAdvancer / PatchLifecycle / SampleBank 等)。登録: Window → MIDI → Visual Scripting → Register Chunity Nodes

Editor 診断 (追加 define 不要) : **Window** → **MIDI** → **Chunity** → **Diagnostics** — ログレベルと統合状態。

Scriptable Audio Generator (追加オプション)

制御パス (Bridge / Mapping / Preset 等) はそのままに、音声出力だけを OnAudioFilterRead から Unity 6.3+ の **Scriptable Generator** (IAudioGenerator) へ差し替えます。メトロノーム等の本体 Scriptable Audio 統合 (FEATURE_SCRIPTABLE_AUDIO) とは別シンボルです。

型	役割
ChuckMainGeneratorDriver	Main VM を AudioSource.generator で駆動
ChuckSubGeneratorDriver	Sub (空間化付き) を Generator で駆動
ChuckAudioOutputMode	Auto / FilterRead / ScriptableGenerator

モード	挙動
Auto (推奨)	対応環境では Generator、それ以外 (WebGL 等) は従来 FilterRead
FilterRead	標準 Chunity と同じ OnAudioFilterRead
ScriptableGenerator	Generator を厳密に使用 (非対応時は警告 1 回のうすフォールバック)

セットアップ:

- 有効化手順 (Scriptable Audio Generator) のパッチ適用、** com.unity.collections ** のインストール、FEATURE_CHUNITY_SCRIPTABLE_AUDIO の追加
- ChuckMainInstance に ChuckMainGeneratorDriver を追加 (Sub 利用時は各 ChuckSubInstance に ChuckSubGeneratorDriver)
- Audio Output Mode を設定 (通常は Auto)
- Play — Driver は Generator 接続中に useBuiltInAudioFilter = false とし、二重再生を防ぐ

運用上の注意:

項目	内容
二重再生禁止	Generator 有効時は FilterRead を無効化。同時駆動しない
Main → Sub 順序	両方 Generator のときは <code>ChuckGeneratorTracker</code> が Main の VM 進行後に Sub を許可。Main が FilterRead のときは Chunity Mixer 順に依存
マイク (adc)	Main Driver が録音バッファをメインスレッドから lock-free リング経由で DSP へ供給
ミュート	<code>AudioSource.mute</code> を尊重。VM 時計は進めつつ出力のみ無音
サンプルレート	Chuck 初期化レートと Generator <code>AudioFormat.sampleRate</code> が異なると警告

制約・ロードマップ

- Chunity ランタイムは **非同梱**。インストールと `Chunity.Runtime.asmdef` と `FEATURE_CHUNITY` の三者が必要
- シンボルのみ有効で Chunity 未導入の場合は型解決エラーになる（MPTK と同様）
- 利用者は常に Chunity 生 API を直接呼べる。本プラグインは MIDI ドメインの薄いブリッジ層
- **未実装（ロードマップ）**：MIDI 2.0 / UMP マッピング、InstanceTarget の String / Once / GetArray / RunFile 引数、Syncer・連想配列・*_AT 書き込み、VM / shred 制御ヘルパ、UGen プローブ、Host Time Advancer 等 — [目次の将来機能](#) を参照
- 詳細手順: [Chunity 統合](#) / [ビルド後の処理](#)

Networking 統合

LAN 上の MIDI イベント / SMF 再生同期。本線は UDP（`FEATURE_MIDI_NETWORK`）。オプションで Mirror・Netcode・[WSNet2](#) 上に同じ Codec を載せます。

コンポーネント	役割
<code>MidiNetworkHub</code> / <code>MidiNetworkClient</code>	UDP 本線
<code>MidiMirrorBridge</code> / <code>MidiNetcodeBridge</code> / <code>MidiWsnet2Bridge</code>	各トランスポート上の Host/Master 権威ブリッジ

手順: [Networking 統合](#)

サンプルシーン

シーン	パス
Animator 統合	Assets/MIDI/Samples/Integrations/Scenes/MidiAnimatorIntegrationSampleScene.unity
Timeline 統合	Assets/MIDI/Samples/Integrations/Scenes/MidiTimelineIntegrationSampleScene.unity
Visual Scripting 統合	Assets/MIDI/Samples/Integrations/Scenes/MidiVisualScriptingIntegrationSampleScene.unity
Input Systemブリッジ	Assets/MIDI/Samples/Integrations/InputSystem/Scenes/InputSystemBridgeSampleScene.unity
ネットワークMIDI (UDP)	Assets/MIDI/Samples/Integrations/Networking/Scenes/MidiNetworkJamSampleScene.unity
ネットワークMIDI (Mirror)	Assets/MIDI/Samples/Integrations/Networking/Scenes/MidiMirrorNetworkSampleScene.unity
ネットワークMIDI (Netcode)	Assets/MIDI/Samples/Integrations/Networking/Scenes/MidiNetcodeNetworkSampleScene.unity
ネットワークMIDI (WSNet2)	Assets/MIDI/Samples/Integrations/Networking/Scenes/MidiWsnet2NetworkSampleScene.unity
Scriptable Audio 統合	Assets/MIDI/Samples/Integrations/ScriptableAudio/Scenes/ScriptableAudioMetronomeSampleScene.unity
Scriptable Audio SMFシーケンス	Assets/MIDI/Samples/Integrations/ScriptableAudio/Scenes/ScriptableAudioSequenceSampleScene.unity
Scriptable Audio UMPシーケンス	Assets/MIDI/Samples/Integrations/ScriptableAudio/Scenes/ScriptableAudioUmpSequenceSampleScene.unity
Chunityブリッジ	Assets/MIDI/Samples/Integrations/Chunity/Scenes/ChunityBridgeSampleScene.unity
Chunityワークフロー	Assets/MIDI/Samples/Integrations/Chunity/Scenes/ChunityWorkflowsSampleScene.unity
Chunityプリセット	Assets/MIDI/Samples/Integrations/Chunity/Scenes/ChunityPresetsSampleScene.unity
ChunityマイクFX	Assets/MIDI/Samples/Integrations/Chunity/Scenes/ChunityMicFxSampleScene.unity
Chunity Generator Workflow	Assets/MIDI/Samples/Integrations/Chunity/Scenes/ChunityGeneratorWorkflowSampleScene.unity

各シーンは仮想 MIDI 入力による ImGui テストに対応しています。詳細な手順は [サンプル](#) を参照してください。

Animator 用 `MidiAnimatorSample.controller` は `Samples/Integrations/Resources/` に同梱されています。再生成する場合は `Window > MIDI > Samples > Generate Integration Sample Assets` を実行してください。

VST3 プラグインホスト（別パッケージ）

VST3 楽器／エフェクトのホスト機能は 本 **MIDI** プラグインには含まれません。MIDI リリース物に `VstHostNative.dll`・VST3 SDK・VST ホスト C# は 同梱されません。

別パッケージを利用してください:

項目	値
表示名	Unity Plugin Host for VST3
UPM / Git URL	<code>https://github.com/kshoji/Unity-VST3-Bridge.git</code>
パッケージ id	<code>jp.kshoji.unity.vst3nativehost</code>

セットアップ・スキャンパス・任意の MIDI 配線は VST 側リポジトリの文書（`Documentation~/usage.md` など）を参照してください。本パッケージは公開 MIDI イベントの購読のみを想定し、`Assets/MIDI` 配下に VST 実装を置かないでください。デスクトップ Player 向け VST MCP は Bridge 側；**Android 実機 MCP は VST3 対象外**です。

VST パッケージ側の任意 MIDI ヘルパー（`FEATURE_MIDI_PLUGIN` が必要）:

機能	コンポーネント
CC / ピッチベンド → VST パラメータ	<code>VstMidiParameterMapping</code> , <code>VstHostMidiParameterMapper</code>
SMF → VST 音源	<code>VstHostSmfLink</code> + <code>SmfPlayer.outputDeviceId</code> → 仮想デバイス → <code>VstHostMidiAdapter</code> （チャンネル振り分け可）
プリセット / A/B	<code>VstPresetAsset</code> , <code>VstPresetBrowser</code> , Window → VST3 Host → Preset Browser

追加の VST パッケージ統合（Timeline / Input System があるとオプションアセンブリが有効化）:

機能	コンポーネント
Timeline パラメータオートメーション	<code>VstParameterTrack</code> / <code>VstParameterClip</code> , <code>VstProgramChangeMarker</code> （ <code>FEATURE_USE_TIMELINE</code> ）
Animator ↔ VST	<code>VstAnimatorMapping</code> , <code>VstAnimatorDriver</code>
Input System → VST	<code>InputSystemToVstBridge</code> （ <code>FEATURE_INPUT_SYSTEM</code> ）または MIDI の <code>InputSystemToMidiBridge</code> → Adapter
エディタツール	Plugin Browser（カテゴリ + Vendor/Tag）, Activity Monitor, Virtual Controller, Project Settings → VST3 Host
プラグインチェーン	<code>VstPluginChain</code> , <code>VstHostChannelRouteSync</code> , <code>VstHostMidiFilterLink</code> , <code>VstHostEventSink</code>
Scriptable Audio（Unity 6.3+）	<code>VstHostGenerator</code> 、 <code>VstHostDspMidiOutBridge</code> → スケジューラの <code>extraTimedMidiOutput</code>
Visual Scripting	VST3 Host ユニット / イベント（ <code>FEATURE_USE_VISUALSCRIPTING</code> ）
ネットワーク MIDI → VST	<code>VstHostNetworkMidiLink</code> （ <code>FEATURE_MIDI_NETWORK</code> ）
Chunity ↔ VST	<code>VstHostChuckEventMidiLink</code> 、 <code>VstHostChuckEffectBridge</code> （ <code>FEATURE_CHUNITY</code> ）

詳細: VST パッケージの `Documentation~/timeline.md` / `animator-input.md` / `editor-tools.md` / `plugin-chain.md` / `scriptable-audio.md` / `visual-scripting.md` / `network-midi.md` / `chunity.md`。

関連ドキュメント

- [ゲームプレイ向けコンポーネント](#) — `SmfPlayer` / `MidiRecorder` / `MidiInputRouter` / `MidiClockSync`
- [SMF ツール](#) — `MidiSequenceAsset` / `UmpSequenceAsset` / `TempoMapExtractor`
- [ユーティリティ](#) — `MidiNoteUtility` / `MidiMessageBuilder`
- [ビルド後の処理](#) — オプション統合シンボルの有効化
- [サンプル](#) — 統合サンプルシーンの手順
- [組み込み済みサードパーティモジュール](#) — 組み込みモジュール（VST ホストは別パッケージ・非同梱）